

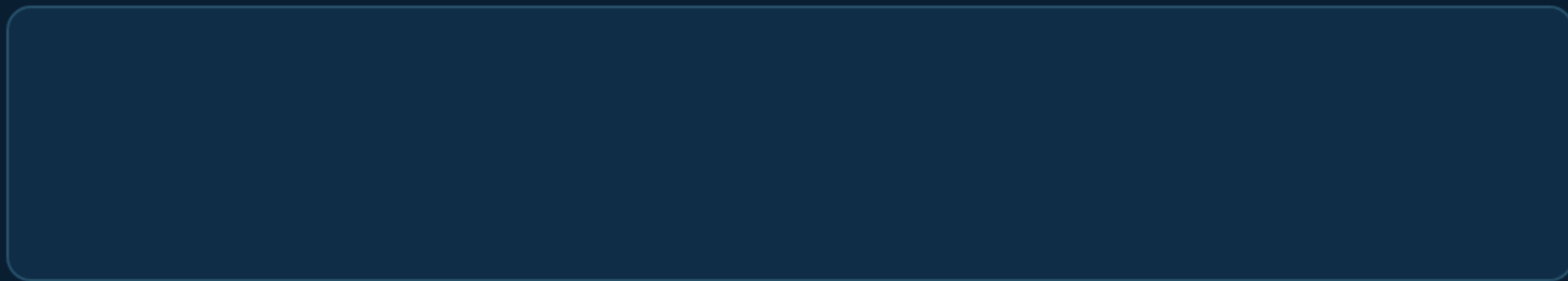
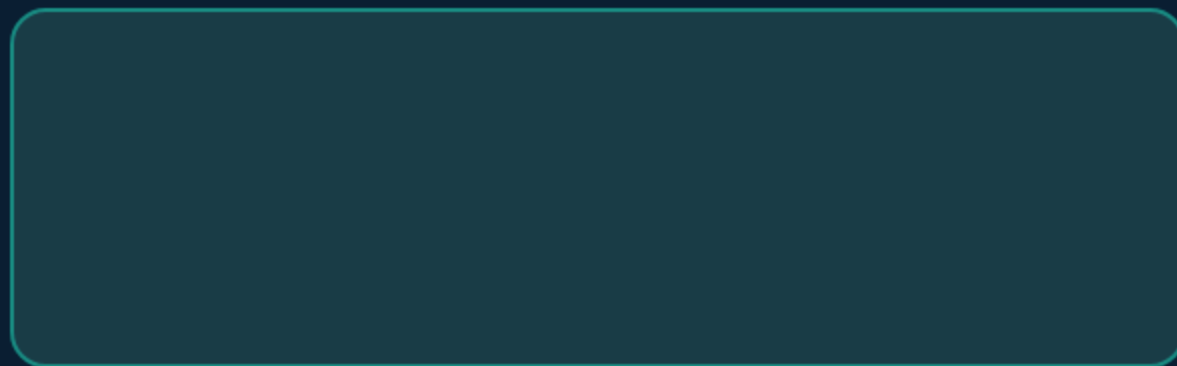
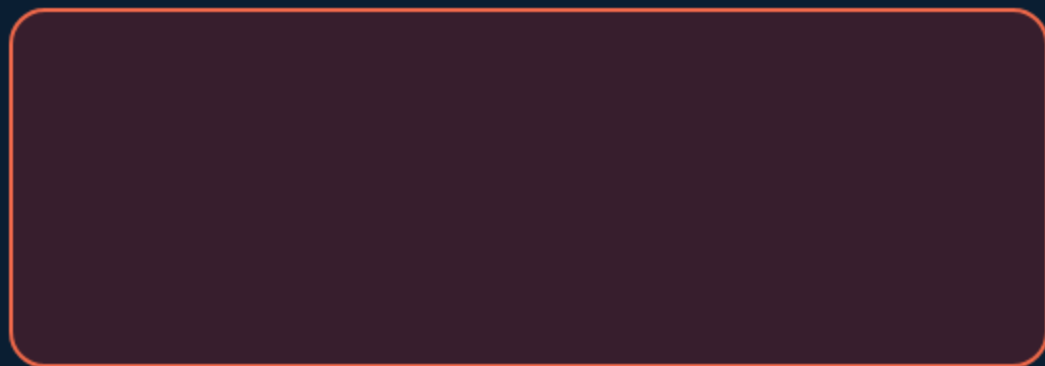
企业真正缺少的， 是选对 Agent 方案的方法。

场景是活分布，样本持续流入。AgentFit 从最简方案开始，在样本上训练，依据证据更新四层方案，迭代到收敛后交付。

方案不是设计出来的，是训练出来的。

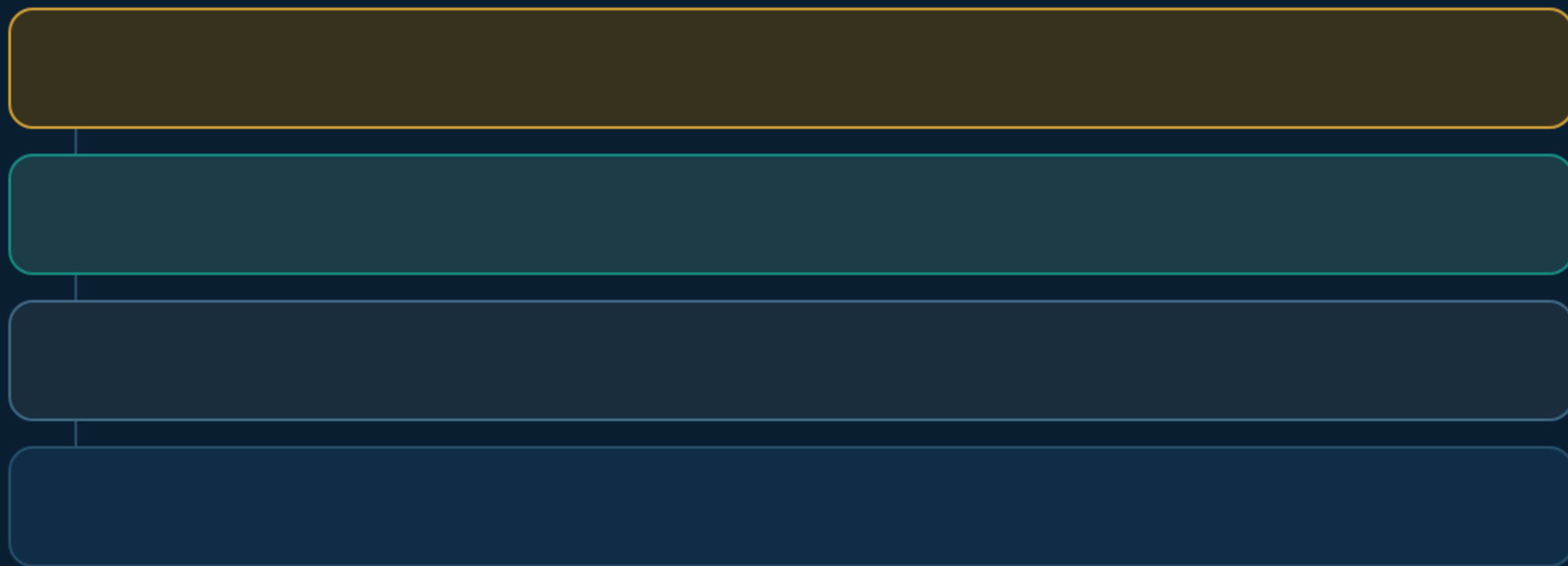
AgentFit: 方案不是设计出来的，是训练出来的。

传统做法"设计→评审→部署"靠经验，AgentFit 用"训练→评估→更新→收敛→部署"靠数据。



四层骨架：Solid → 能力 → 知识 → 拓扑

每一层只做一件事。层间逐层调用，同层禁止执行时互调。



铁律：L3 不能直接调 L1 · 同层禁止执行时互调 · 每个能力必须追溯到物理基础设施

存在依赖：每个能力追溯到物理基础设施。

不能凭空创造。任何一层声明能力，下层必须已提供支撑。

声明链（自顶向下）

L4 "我有诊断 Agent"

→ L3 必须已有 L3 "诊断排查链"

→ L2 必须已有 L2 "safe_get_device_state"

训练循环：执行 → 归因 → 更新 → 回归 → 迭代。

九步闭环。每步产出结构化数据，训练日志记录一切。

①

前向执行

取一批样本用当前方案执行

②

损失归因

自底向上L1→L4 + 因果性验证

③

聚合分析

损失模式 + 正则指标计算

④

更新建议

分层建议 + λ 调节

⑤⑥

人审

审核方向和安全边界

自底向上归因：错在哪层、为什么、怎么修。

从 L1 开始逐层检查。找到异常后验证因果性，不是"找到即停"。

归因流程

Step 1 → L1 "原子存在吗？"

缺 → 验证因果 → 是根因则停 / 否则标记附带问题

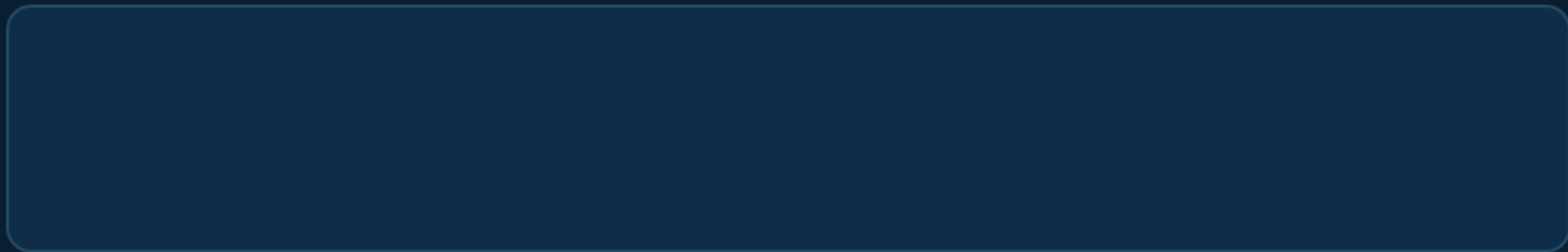
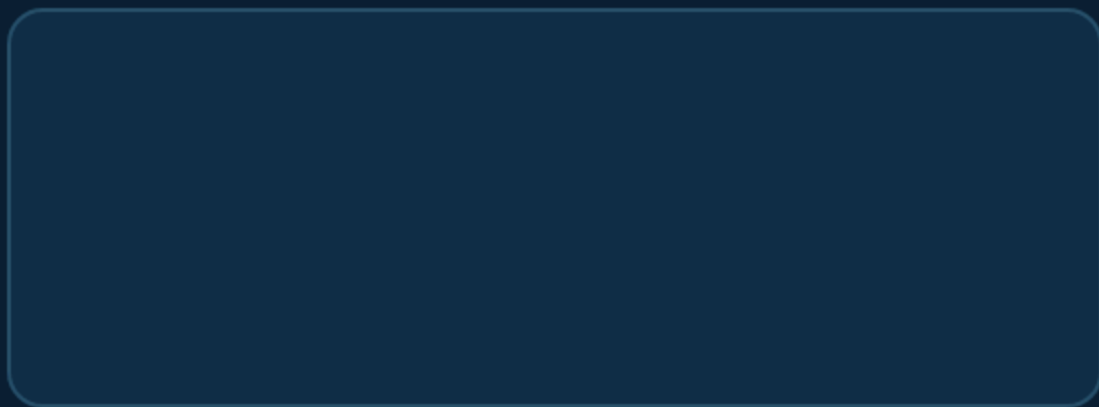
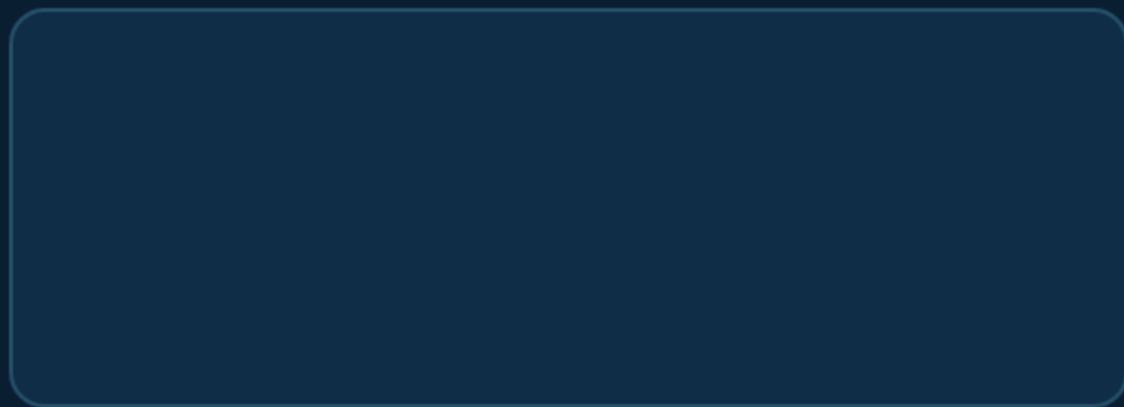
Step 2 → L2 "封装正确吗？"

缺 → 验证因果 → 是根因则停 / 否则标记附带问题

Step 4 → L4 "架构适合吗？"

正则约束：可维护性从口号变成数字

16 指标 × 4 层。每层正则 = $\max(0, \text{worst_violation_ratio})$ 。λ 两级调节。



可审计：训练日志（哈希链保护）

每轮追加不可篡改记录。任何人可回答"为什么改、改了什么、有没有改坏"。

训练日志条目

previous_hash : abc123...

可追溯：失败 → 根因 → 修复 → 验证

每个失败样本从 L1 到 L4 全链路归因，附带因果性验证。

损失轨迹示例

样本 #42 失败 (复合根因：飞行模式 + 漫游)

L3：路由器处理了漫游 (主因)，没处理飞行模式 (次因)

因果性验证："修复路由后能通过吗？" → 是 → 确认根因 = L3

可量化：通过率 / 成本 / 正则值 / 回归率

每轮训练自动产出全部指标。数字说话，不靠感觉。

85%

通过率

训练 3 轮后

\$0.006

成本/样本

vs 人工 \$15

100%

回归通过率

必须 100%

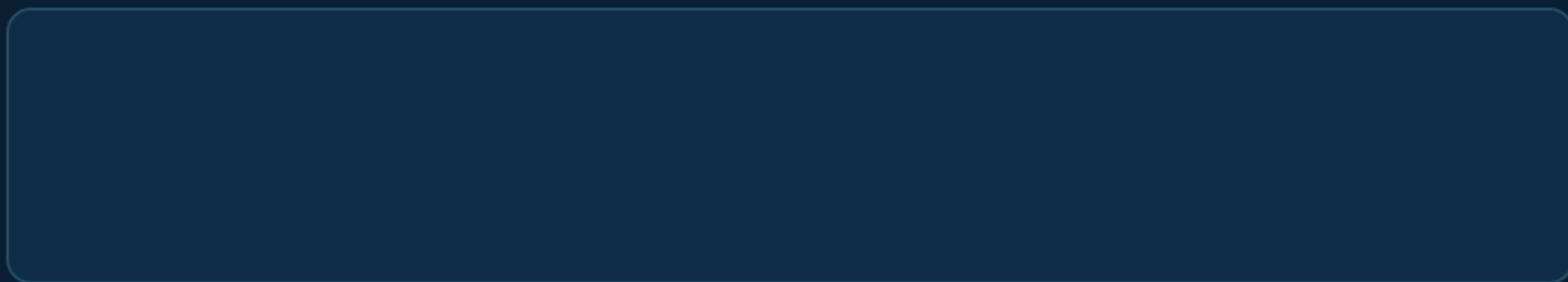
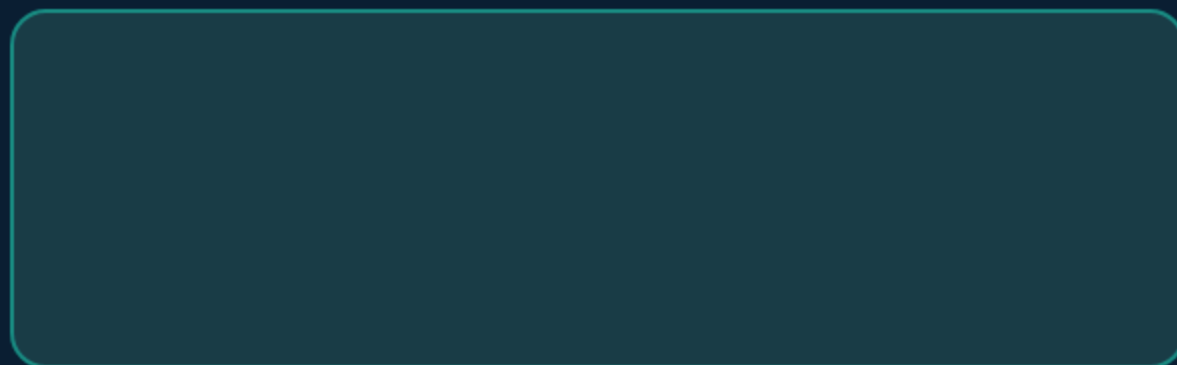
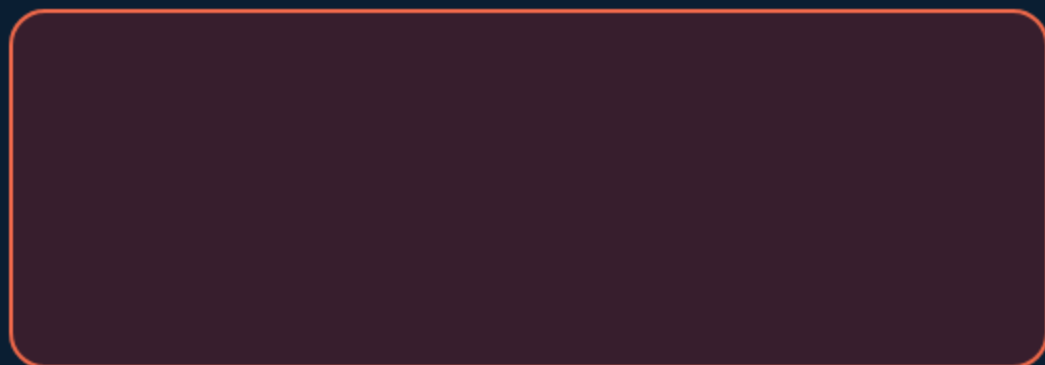
0.08

L3 正则值

阈值 0.10 内

Telecom 实测：80% baseline → AgentFit 目标

τ^2 -bench telecom 基准已跑通，10 样本实测通过率 80%，成本 \$0.006/样本。



五种合法交付：包括"拒绝自动化"

"不自动化"和"自动化"一样是合法结论，且有证据支撑。

全自动

AI 全权处理
通过率 $\geq$ 阈值
风险可控

部分自动

AI 常规 + 人关键
多数自动
少数需人

降级

简化版 + 条件限制
预算/能力有限

保留人工

不值得自动化
成本 > 人工

拒绝自动化

不该用 AI
通过率太低
风险太大

AgentFit 不预设 AI 总是更好。Fit = 有证据地选对方案。

四层骨架完整定义

详见 `agentfit-skeleton.md`

反向归因算法伪代码

详见 `agentfit-skeleton.md`

正则指标完整清单

详见 agentfit-skeleton.md

测试场景执行方案

详见 test-scenario.md

开源承诺 (MIT License)

MIT License · 全部开源
